

How Computers Calculate Square Root and Flop/s Performance

Aiichiro Nakano

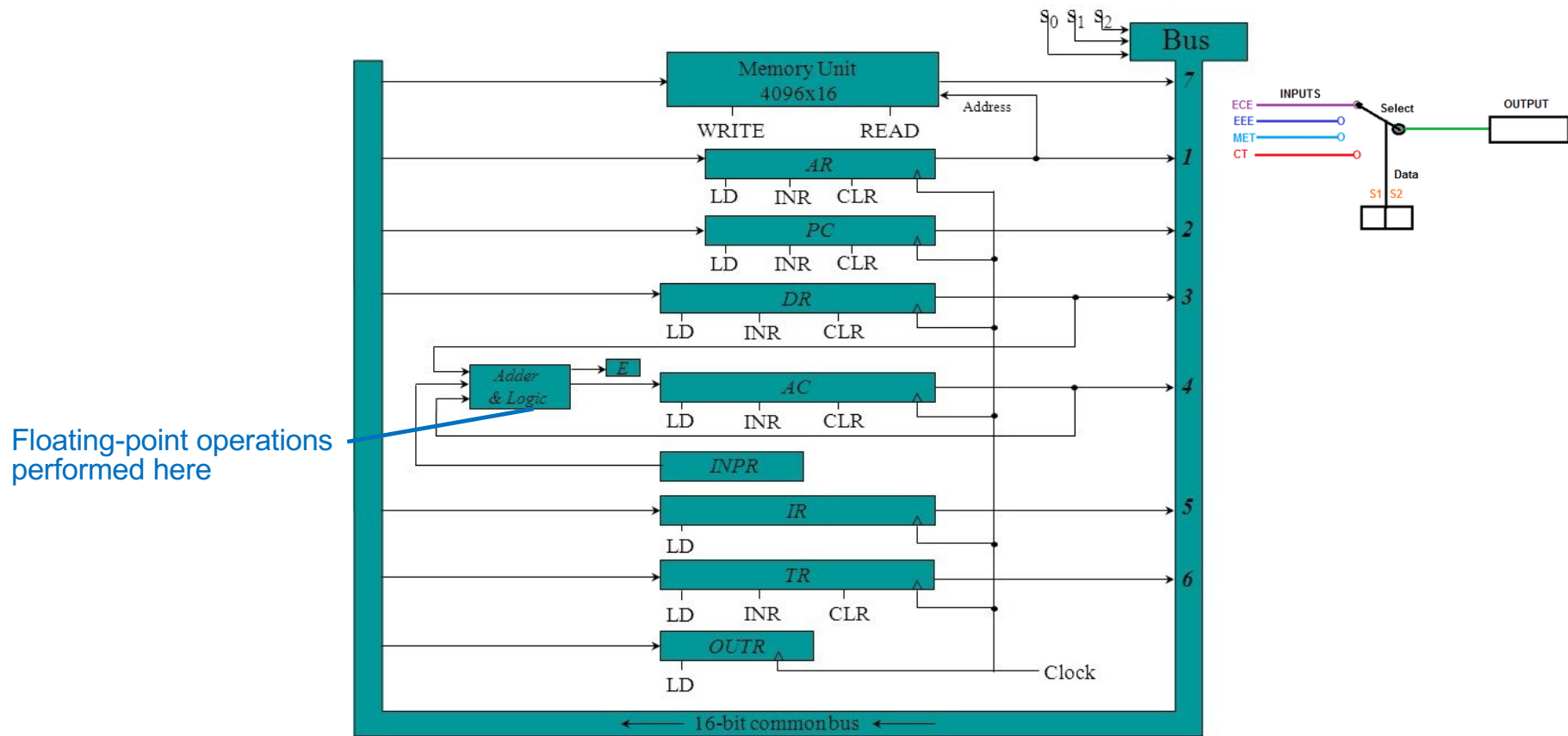
*Department of Computer Science
Department of Physics & Astronomy
Department of Quantitative & Computational Biology
University of Southern California*

Email: anakano@usc.edu

- Demystifying mathematical-function black box
- What to count for flops for `sqrt()` in `lmd_sqrt_flop.c`?
- New perspective on Flop/s performance



Basic Computer Architecture



Computer System Architecture, Mano, Copyright (C) 1993 Prentice-Hall, Inc.

M. M. Mano, *Computer System Architecture* (Prentice-Hall)

FLOATING-POINT UNIT DESIGN
USING TAYLOR-SERIES EXPANSION ALGORITHMS

by

Taek-Jun Kwon

Thesis Proposal

UNIVERSITY OF SOUTHERN CALIFORNIA
ELECTRICAL ENGINEERING

September 2006

How Time Consuming Is SQRT()?

Table 1.1 Summary of prototype FPUs

Design	Cycle time (ns)	Latency/Throughput (cycles/cycles)			
		$a \pm b$	$a \times b$	$a \div b$	$\sqrt{a}$
DEC 21164 Alpha AXP	2.0	4/1	4/1	22-60/22-60	N/A
Hal Sparc64	6.49	4/1	4/1	8-9/7-8	N/A
HP PA 7200	7.14	2/1	2/1	15/15	15/15
HP PA 8000	5.0	3/1	3/1	31/31	31/31
IBM RS/6000 Power 2	14.0	2/1	2/1	16-19/15-18	25/24
Intel Pentium	5.0	3/1	3/2	39/39	70/70
Intel Pentium Pro	7.52	3/1	5/2	30/30	53/53
MIPS R8000	13.3	4/1	4/1	20/17	23/20
MIPS R10000	3.64	2/1	2/1	18/18	32/32
PowerPC 604	5.56	3/1	3/1	31/31	N/A
PowerPC 620	7.5	3/1	3/1	18/18	22/22
Sun SuperSparc	16.7	3/1	3/1	9/7	12/10
Sun UltraSparc	4	3/1	3/1	22/22	22/22

- **Latency:** How many clock cycles to complete 1 operation
- **Throughput:** Cycles before the next operation can be issued

Hardware Implementation of SQRT()

- **Newton-Raphson method**

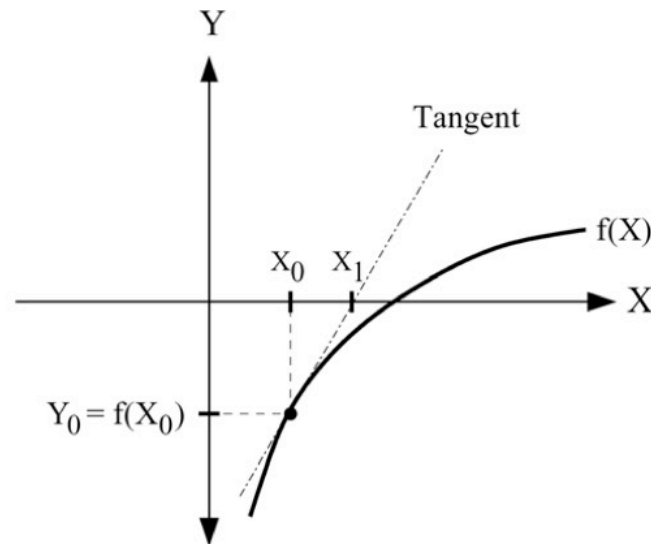


Figure 2.1 Newton-Raphson algorithm for finding the root of $f(x)$

- **Series expansion**

$$\sqrt{b} \approx Y_0 \left\{ 1 - \frac{1}{2} \left(1 - \frac{b}{Y_0^2} \right) - \frac{1}{8} \left(1 - \frac{b}{Y_0^2} \right)^2 - \frac{1}{16} \left(1 - \frac{b}{Y_0^2} \right)^3 - \frac{15}{128} \left(1 - \frac{b}{Y_0^2} \right)^4 \right\}$$

Simple Sqrt() Routine

- Initial Guess**

$$r = s^{\frac{1}{2}}$$

$$\approx f(s) = c_0 + c_1 s + c_2 s^2 + c_3 s^3$$

$$= c_0 + s \times (c_1 + s \times (c_2 + s \times c_3))$$

where $0.1 < r^2 < 1.0$

$$c_0 = 0.188030699; c_1 = 1.48359853$$

$$c_2 = -1.0979059; c_3 = 0.430357353$$

- Newton-Raphson Refinement**

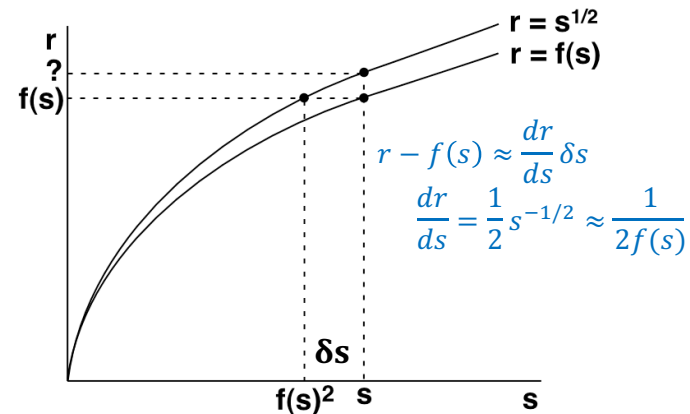
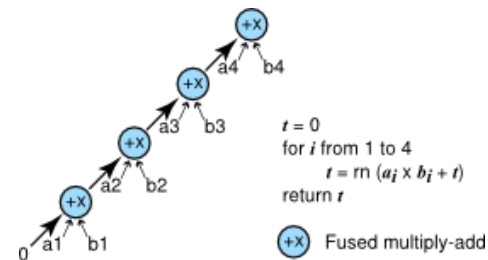
$$\delta s \leftarrow s - f(s)^2$$

$$r \leftarrow f(s) + \delta s / 2 f(s)$$

M.P. Allen & D.J. Tildesley, *Computer Simulation of Liquids* (Oxford Univ. Press, Oxford, 1987) p.143

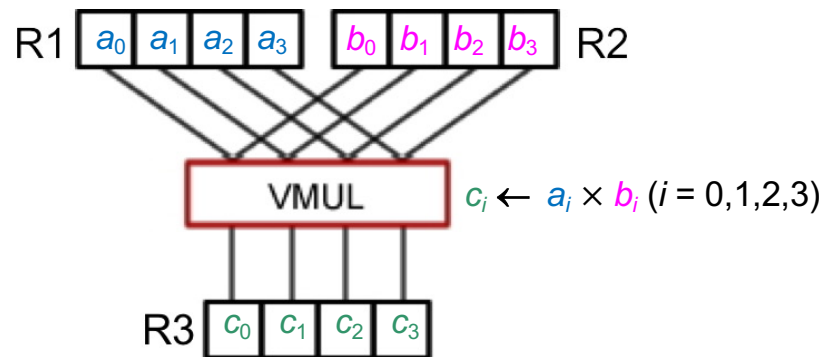
Fused multiply-add (FMA) unit

$a \leftarrow a + b \times c$
with 1-cycle throughput



SIMD/Vector Operation

- Each FMA operation can work on a set of multiple operands concurrently
- Single-instruction multiple-data (SIMD) parallelism: An arithmetic operation is operated on multiple operand-pairs stored in vector registers, each of which can hold multiple double-precision numbers.



Example: Vector multiplier (VMUL) loads data from two vector registers, R1 and R2, each holding 4 double-precision numbers, concurrently performs 4 multiplications, and stores the results on vector register R3.

New Perspective: Flop/s Performance & Precision

- High performance computing (HPC) is at a historic crossroads, where traditional modeling & simulation applications may not survive

Deelman et al., Science 387, 829 ('25)

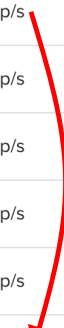
- Computers are increasingly more *heterogeneous* by integrating different functional units, focusing on *low-precision* arithmetic to serve market-dominating AI applications

Razakh et al., IEEE/ACM SC ('25), DOI: 10.1145/3712285.3771785

Heterogeneity/low-precision challenge!

General matrix-matrix multiply (GEMM) performance on Aurora GPU

	One Tile	Full Node	Scaling
DGEMM 64 bits	15 TFlop/s	179 TFlop/s	11.9
SGEMM 32 bits	22 TFlop/s	258 TFlop/s	11.7
HGEMM 16 bits	263 TFlop/s	2606 TFlop/s	9.9
BF16GEMM	273 TFlop/s	2645 TFlop/s	9.7
TF32GEMM	110 TFlop/s	1311 TFlop/s	11.9
I8GEMM 8 bits	577 TFlop/s	5394 TFlop/s	9.4



cf. Atchley et al., Computer 58(7), 110 ('25)

Precision vs. Flop/s Tradeoff

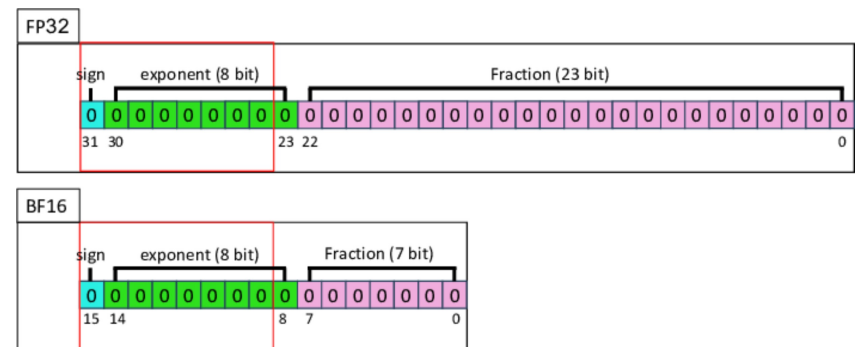
Tale of three Gordon Bell prize finalists in 2025: Dramatic effect of floating-point (FP) & brain-float (BF) precisions on Flop/s (floating-point operations per second) performance on Aurora supercomputer

- **Simulation:** “Advancing quantum many-body GW calculations on exascale supercomputing platforms,” Zhang (USC) et al.
0.71 Exaflop/s on 9,600 nodes (FP64)
- **Multi-Exaflop/s simulation:** “Multiscale light-matter dynamics in quantum materials: from electrons to topological superlattices,” Razakh (USC) et al.
1.87 Exaflop/s on 10,000 nodes (hybrid FP64/FP32/BF16)
- **Machine learning (ML):** “AERIS: Argonne earth systems model for reliable and skillful predictions,” Hatanpaa (Argonne) et al.
11.2 Exaflop/s on 10,080 nodes (FP32/BF16)

Hybrid-precision accumulation/
matrix-multiplication

$$\overbrace{\mathbf{M} += \sum_i}^{\text{FP32}} \overbrace{\mathbf{L}_i \times \mathbf{R}_i}^{\text{BF16}}$$

BF16 has the same exponent range as FP32 but with less precision (fraction)



Breaking the simulation-ML dichotomy